RVS UNIVERSITY

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

PROJECT REPORT

RESTAURANT MANAGEMENT SYSTEM

A Desktop Application for Centralized Restaurant Operations, Order Tracking and Billing

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in Electronics and Communication Engineering

Guided by

Naveen Sir

S.No.	Name	Roll Number
1	Hemani	23781A04C6
2	Sumalatha	23781A04D5
3	Lokesh	23781A04E4
4	Shiva Shankar	23781A04E7
5	Chaitanya	23781A04F0
6	Kishore	23781A04F3
7	Manoj	23781A04F5
8	Karthik	23781A04F6

IV B.Tech

CERTIFICATE

This is to certify that the project report entitled "Restaurant Management System" has been successfully carried out by the below-mentioned students of IV B.Tech, Department of Electronics and Communication Engineering, RVS University, under the guidance of Naveen Sir. The system was originally conceived as a restaurant billing utility and has since been extended into a complete desktop-based Restaurant Management System covering menu management, table management, order management, a kitchen display workflow, billing and operational reporting. The work presented in this report is submitted as part of the academic requirements of the Bachelor of Technology programme.

Guide: Naveen Sir

Department of Electronics and Communication Engineering

Faculty Guide Signature

_____

Naveen Sir

Date: _____

DECLARATION

We hereby declare that the project report entitled "Restaurant Management System" is our original academic work, carried out as a group project under the guidance of Naveen Sir, Department of Electronics and Communication Engineering, RVS University. The information presented in this report is prepared for academic purposes and has not been submitted elsewhere for the award of any degree or diploma.

Project Team

S.No.	Name	Roll Number
1	Hemani	23781A04C6
2	Sumalatha	23781A04D5
3	Lokesh	23781A04E4
4	Shiva Shankar	23781A04E7
5	Chaitanya	23781A04F0
6	Kishore	23781A04F3
7	Manoj	23781A04F5
8	Karthik	23781A04F6

ABSTRACT

The Restaurant Management System is a desktop-based graphical application developed using the C programming language (C99 standard) and the GTK3 toolkit to provide centralized management of day-to-day restaurant operations. What began as a simple restaurant billing utility has been expanded into a complete operational system that brings together a dashboard, menu management, table management, order management, a kitchen display workflow, billing and reporting under a single graphical interface.

In a traditional restaurant environment, menus, table status, customer orders, kitchen coordination and bills are handled manually or across disconnected tools, which leads to inconsistent information, communication gaps between front-of-house staff and the kitchen, and errors in bill calculation. The Restaurant Management System addresses these issues by giving staff a single application in which the menu can be maintained, tables can be assigned, reserved or released, orders can be created and tracked through defined preparation stages, and bills can be generated with discount and tax calculations applied automatically.

The application provides an executive dashboard summarizing total and active orders, table availability and occupancy, average order value and revenue realized; a menu management screen for adding, editing, enabling/disabling and searching food items; a table management screen showing the live status of every table; an order and Kitchen Display System that tracks orders through Pending, Preparing, Ready, Served, Completed and Cancelled states; a billing module that computes subtotal, discount, tax and grand total; and a reporting module that surfaces operational statistics and top-selling items.

The project demonstrates the practical application of core and intermediate C programming concepts, including structures, enumerations, arrays, functions, conditional statements, loops, modular file organization, and event-driven GUI programming using GTK3 callbacks. This report presents the objectives, existing system, proposed system, technologies used, system modules, functional and non-functional requirements, architecture, data model, workflow, methodology, source code implementation, testing, screenshots of the working application, advantages, limitations, future scope and conclusion of the Restaurant Management System.

TABLE OF CONTENTS

1. INTRODUCTION

A restaurant involves several interconnected activities that must work together smoothly for daily operations to run without friction. These include maintaining the menu and item pricing, tracking which tables are available, occupied or reserved, recording customer orders, coordinating those orders with the kitchen, tracking the preparation status of each order, generating an accurate bill, and monitoring revenue and performance over time. When these activities are handled manually or through disconnected paper-based methods, restaurants frequently experience delays, communication gaps between staff and the kitchen, inconsistent pricing, and billing errors, particularly during busy hours.

The Restaurant Management System has been developed to bring these activities together into a single centralized desktop application. The system began as a smaller console-based Restaurant Billing System focused only on bill calculation. It has since been substantially redeveloped and expanded into a full graphical Restaurant Management System, built using the C programming language (C99) and the GTK3 GUI toolkit, that manages the restaurant's menu, tables, orders, kitchen workflow, billing and operational reporting through one consistent interface.

This project demonstrates how core programming concepts such as structures, enumerations, arrays, functions, conditional statements and loops can be combined with event-driven GUI programming to build a practical, real-world desktop application suitable for a B.Tech academic project.

2. PROBLEM STATEMENT

Traditional, manual restaurant operations face a number of recurring difficulties, including:

- Menu items and prices are maintained on paper or verbally communicated, making updates slow and inconsistent.
- Table availability is tracked informally, making it difficult for staff to know at a glance which tables are free, occupied or reserved.
- Reservations are noted manually and can be forgotten or double-booked.
- Customer orders are handwritten, which is slow during busy periods and prone to transcription errors.
- Communication gaps exist between front-of-house staff and the kitchen regarding what has been ordered and its preparation status.
- There is no structured way to track whether an order is pending, being prepared, ready, or already served.
- Bills are calculated manually, which increases the risk of calculation errors, particularly when discounts and taxes must be applied.
- Order history is difficult to maintain, making it hard to review past orders or analyse performance.
- There is no centralized source of operational information such as today's revenue, occupancy or order volume.
- Restaurant performance, such as which items sell best, is difficult to analyse without consolidated data.

These difficulties collectively point to the need for a centralized, computerized Restaurant Management System that can maintain the menu, manage tables, record and track orders through the kitchen workflow, generate accurate bills, and provide operational visibility to staff and management.

3. OBJECTIVES

- To develop a desktop-based Restaurant Management System using C and GTK3.
- To provide a centralized graphical interface for restaurant operations.
- To manage restaurant menu items, categories and prices.
- To manage table availability, occupancy and reservations.
- To create and manage customer orders, including items and quantities.
- To track order preparation through a Kitchen Display System.
- To calculate and generate bills, including discount and tax computation.
- To maintain order-related information for reference and reporting.
- To provide dashboard statistics and operational reports, including top-selling items.
- To demonstrate practical application of C99 and GTK3 in a real-world scenario.
- To reduce manual effort and improve the organization of restaurant operations.

4. EXISTING SYSTEM

In the traditional, manual approach to restaurant management, the menu is maintained on printed cards or boards and updated by hand whenever prices or items change. Table availability is tracked informally by restaurant staff walking the floor or relying on memory, and reservations are noted in a physical register or verbally communicated between staff. Customer orders are written down on paper, and the same order slip is often carried to the kitchen, which makes it difficult to track preparation status or notify staff when an order is ready. Bills are calculated manually, item by item, with discounts and taxes applied by hand.

This existing approach has several drawbacks: menu and price information can become inconsistent across staff; table status is not visible at a glance; reservations can be missed or double-booked; order records depend on legible handwriting and are easy to lose; there is no structured tracking of order preparation stages; bill calculations are vulnerable to human error; historical order data is rarely retained in a usable form; and there is no consolidated view of restaurant performance such as revenue, occupancy or best-selling items. These limitations motivate the development of a computerized Restaurant Management System.

5. PROPOSED SYSTEM

The proposed Restaurant Management System is a desktop GUI application, built with C99 and GTK3, that centralizes restaurant operations into seven connected modules: Dashboard, Menu Management, Table Management, Order Management, Kitchen Display System, Billing, and Reports. Each module addresses a specific operational need, and the modules are linked so that actions in one module are reflected in the others — for example, assigning a table to an order updates that table's status, and an order's progress through the Kitchen Display System is reflected in dashboard statistics.

The overall conceptual workflow of the proposed system is summarized below:

Dashboard → Menu Management → Table Management → Create Order → Kitchen Display System (Pending → Preparing → Ready → Served) → Billing → Reports / Analytics

Menu items maintained in the Menu Management module are available for selection when an order is created. Tables maintained in the Table Management module are assigned to orders, which links a table's status directly to the order occupying it. Once an order is created, it is tracked through the Kitchen Display System as it moves through preparation stages. When an order is served, its items and totals feed into Billing, and completed orders contribute to the statistics shown on the Dashboard and Reports modules. This interconnected design allows staff to manage the full lifecycle of a customer visit — from seating to billing — from within a single application.

6. TECHNOLOGIES USED

Technology	Details
Programming Language	C
Language Standard	C99
GUI Toolkit	GTK3
Compiler	GCC
Application Type	Desktop GUI Application

Role of Each Technology

- C (C99): Provides the core language used to implement the application's data structures, business logic and control flow, including menu, table, order, billing and statistics handling.
- GTK3: Provides the graphical toolkit used to build the desktop interface — windows, tabs, buttons, tables/grids and dialogs — and to respond to user actions through event callbacks.
- GCC: Used to compile the C source files into the desktop executable.

Programming Concepts Applied

- Structures — to model FoodItem, Table, Order, OrderItem, DashboardStats and TopItemStat entities.
- Enumerations — to represent fixed sets of states such as TableStatus and OrderStatus.
- Arrays — to hold collections such as menu items, tables, orders and the items within an order.
- Functions — to implement modular operations for menu, table, order, billing and reporting logic.
- Conditional statements and loops — to implement validation, searching, filtering and status-based logic.
- Modular programming — the application is organized into separate source and header files by concern (data/logic vs. GUI).
- GUI callbacks — GTK3 signal handlers connect user interface events (button clicks, selections) to the underlying C logic.
- Input validation — used when adding menu items, creating orders and updating quantities.

7. SYSTEM MODULES

7.1 Dashboard and Analytics

The Dashboard module, shown in the application as the "Restaurant Executive Dashboard & Analytics" screen, gives staff and management a high-level operational snapshot. It displays Total Orders, Active Orders, Available Tables, Occupied Tables, Reserved Tables, Table Occupancy percentage, Average Order Value, and Today's Total Revenue Realized. It also provides Quick Management Actions — single-click shortcuts to Menu Management, Table Management, Create/View Orders, Kitchen Display, and Billing & Receipt — so staff can move directly into the relevant module. The figures shown on the dashboard in the current build (for example, 20 total orders and 40.0% table occupancy) are demonstration/sample data used to illustrate how the application behaves, not real business figures.

7.2 Menu Management

The Menu Management module allows staff to maintain the restaurant's food items. Each item has an ID, name, category, price and availability status. The screen supports searching items by name, filtering by category, adding a new item, editing an existing item, enabling or disabling an item's availability, and deleting an item. The demonstration menu includes starters such as Veg Manchurian, Paneer 65, Paneer Tikka, Gobi 65, French Fries, Chicken Tikka, Chicken 65 and Chicken Manchurian, along with soups such as Tomato Soup and Sweet Corn Soup, with prices displayed exactly as configured in the application (for example, Veg Manchurian at Rs. 140.00 and Chicken 65 at Rs. 200.00).

7.3 Table Management

The Table Management module, presented as the "Table Management & Floor Seating Overview" screen, shows every table's ID, seating capacity and current status — Available, Occupied or Reserved. For an occupied table, the module shows the associated order number and customer name and offers a View Order action and a Release action. For a reserved table, it shows the name the table is reserved for and offers Occupy Guest and Cancel Reservation actions. For an available table, it offers Assign Table and Reserve actions. The demonstration data includes tables T01 through T10 with varying seating capacities and statuses; these are example tables used to demonstrate the module and not a real restaurant floor plan.

7.4 Order Management

The Order Management module allows staff to create a new order against a selected table (or as a walk-in order), record the customer's name, and add food items with quantities drawn from the active menu. Each added item is stored as an order line with its own computed amount (price multiplied by quantity). Staff can update item quantities, remove items from an order, and view an order's current status. Orders are retained so that their history and totals remain available for billing and reporting.

7.5 Kitchen Display System

The Kitchen Display System tracks each order through a defined sequence of preparation states implemented in the application as an enumeration: PENDING, PREPARING, READY, SERVED, COMPLETED and CANCELLED. This gives kitchen and front-of-house staff a shared, structured view of exactly where each order stands, replacing informal verbal communication with a trackable status. The

current implementation manages this status within the desktop application itself; it does not involve separate networked kitchen-display hardware.

7.6 Billing

Billing is one module within the broader Restaurant Management System rather than the system's sole purpose. For each order, the application computes a subtotal from the order's items, applies a discount at a rate of 5% (DISCOUNT_RATE = 0.05) and a tax at a rate of 5% (TAX_RATE = 0.05) as defined in the source code, and produces a grand total combining the subtotal, discount and tax. The system does not currently include online or digital payment integration.

7.7 Reports and Analytics

The Reports and Analytics module draws on the application's DashboardStats structure to present operational metrics: total orders, active orders, completed orders, pending orders, preparing orders, ready orders, served orders, cancelled orders, available tables, occupied tables, reserved tables, today's revenue, average order value and occupancy rate. It also draws on a TopItemStat structure to present top-selling items by total quantity sold and total revenue generated, giving staff visibility into which menu items are performing best.

8. FUNCTIONAL REQUIREMENTS

ID	Requirement
FR1	Display the restaurant dashboard with key operational statistics.
FR2	Display and manage menu items.
FR3	Add new menu items.
FR4	Update existing menu items.
FR5	Delete menu items.
FR6	Enable or disable menu item availability.
FR7	Search and filter menu items by category.
FR8	Display all restaurant tables and their status.
FR9	Assign a table to an order.
FR10	Reserve a table for a customer.
FR11	Release an occupied table.
FR12	Create a customer order.
FR13	Add items to an order.
FR14	Update item quantities within an order.
FR15	Remove items from an order.
FR16	Update the status of an order.
FR17	Track kitchen preparation status of orders.
FR18	Calculate order totals (subtotal, discount, tax, grand total).
FR19	Apply the discount and tax logic implemented in the application.
FR20	Generate and display billing information.
FR21	Calculate dashboard statistics.
FR22	Calculate top-selling item information.
FR23	Persist restaurant data where implemented by the application.

9. NON-FUNCTIONAL REQUIREMENTS

- Usability — the GTK3 interface should be clear and intuitive for restaurant staff with minimal training.
- Accuracy — bill, discount, tax and statistical calculations should be correct and consistent.
- Reliability — the application should behave consistently for repeated use across a working session.
- Maintainability — the C source code is organized into logical modules (data/logic and GUI) so it can be understood and extended.
- Performance — dashboard and menu/table/order operations should respond without noticeable delay for the expected data volumes.
- Portability — the application is built on standard C99 and GTK3, which are available on common desktop Linux environments.
- Readability — the codebase uses descriptive structure and function names to aid understanding.
- Modularity — functionality is separated by concern (menu, table, order, dashboard) to keep modules independent.
- Error handling — operations such as adding menu items or updating orders validate input and report errors through dedicated error-message parameters.
- Interface clarity — status information (table state, order state) is presented with clear, consistent visual indicators.

10. SYSTEM ARCHITECTURE

The Restaurant Management System is organized around a central set of modules coordinated through a shared data model, as shown below.

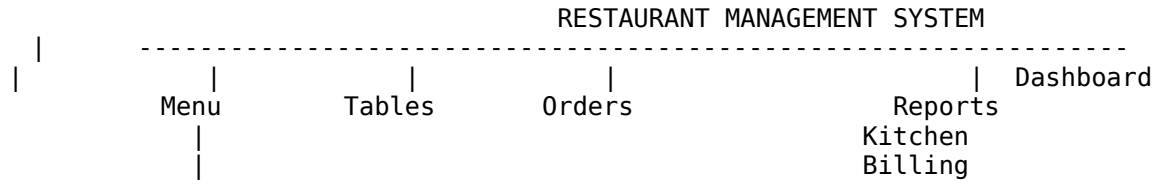

Figure: Conceptual module architecture of the Restaurant Management System

Source Code Organization

The application's source code is organized into the following files, separating core data and business logic from the GTK3 presentation layer:

File	Purpose
main.c	Application entry point; initializes the system and launches the GTK3 interface.
restaurant.h	Declares the core structures, enumerations and function prototypes for menu, table, order and statistics logic.
restaurant.c	Implements the restaurant/menu/table/order business logic declared in restaurant.h.
gui.h	Declares GUI-related types and function prototypes.
gui.c	Implements the GTK3 graphical interface, screen layouts and user-interaction callbacks.

This separation follows a straightforward layered approach — a data/logic layer (restaurant.h / restaurant.c) and a presentation layer (gui.h / gui.c) coordinated by main.c — rather than a formally named architectural pattern such as MVC.

11. DATA MODEL / CORE STRUCTURES

The application's data model is built entirely from C structures and enumerations declared in restaurant.h. The key entities are summarized below.

FoodItem

Represents a single menu item, with fields id, name, category, price and available (1 = available, 0 = unavailable).

Table

Represents a physical restaurant table, with fields id, capacity, status (a TableStatus value), current_order_id (the order currently occupying the table, or 0) and reserved_for (the customer name when reserved).

OrderItem

Represents one line within an order, holding the FoodItem selected, the quantity ordered, and the computed amount (price multiplied by quantity).

Order

Represents a complete customer order, with fields order_id, table_id, customer_name, an array of OrderItem entries with item_count, an OrderStatus value, subtotal, discount_rate, discount_amount, tax_rate, tax_amount, grand_total and a timestamp.

DashboardStats

Stores restaurant-level operational metrics — counts of orders by status, table counts by status, today's revenue, average order value and occupancy rate — used to populate the Dashboard and Reports screens.

TopItemStat

Stores top-selling item information — item ID, item name, total quantity sold and total revenue generated — used by the Reports module to identify best-performing menu items.

12. SYSTEM DESIGN AND WORKFLOW

The overall workflow followed by a staff member using the Restaurant Management System is summarized below.

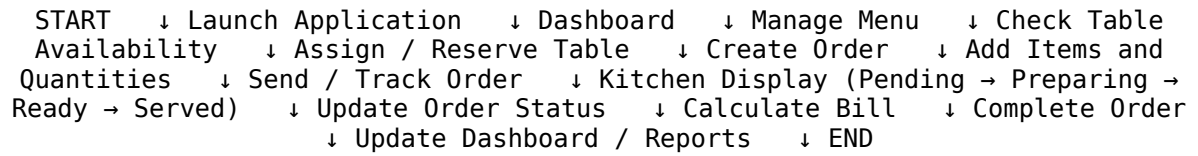

Figure: End-to-end operational workflow of the Restaurant Management System

The workflow begins at the Dashboard, which staff can use to jump into any module. Before creating an order, staff typically confirm the menu is current and check table availability. A table is assigned or reserved, an order is created against that table (or as a walk-in), and items with quantities are added from the menu. As the kitchen works on the order, its status is updated through the Kitchen Display System stages. Once an order is served, billing figures — subtotal, discount, tax and grand total — are calculated, and the completed order updates the statistics shown on the Dashboard and Reports modules.

13. METHODOLOGY

- Requirement identification — understanding the operational needs of a restaurant beyond simple billing.
- Module identification — dividing the system into Dashboard, Menu, Tables, Orders, Kitchen, Billing and Reports.
- Data structure design — defining FoodItem, Table, Order, OrderItem, DashboardStats and TopItemStat in C.
- GUI planning — laying out the GTK3 screens and navigation between modules.
- C implementation — writing the business logic functions for menu, table and order handling.
- GTK3 interface development — building windows, tabs, tables/grids and dialogs and wiring up event callbacks.
- Menu, table and order integration — connecting table assignment and order creation.
- Kitchen status implementation — implementing the OrderStatus state progression.
- Billing implementation — implementing subtotal, discount, tax and grand-total calculation.
- Dashboard and report calculations — implementing DashboardStats and TopItemStat computation.
- Testing — verifying each module's behaviour with representative data.
- Debugging — resolving issues identified during testing.
- Final validation — confirming the application behaves correctly end-to-end before submission.

14. SOURCE CODE IMPLEMENTATION

The following snippets are taken directly from the project's restaurant.h header file, which declares the core data structures, enumerations and business-logic interface used throughout the application. Full source listings are not reproduced here; these snippets illustrate the key design elements referenced elsewhere in this report.

Snippet 1 — Core Data Structures (FoodItem, Table, OrderItem, Order)

```
/* Food Item structure */
typedef struct {
    int id;
    char name[60];
    char category[30];
    double price;
    int available; /* 1 = Available, 0 = Unavailable */
} FoodItem;

/* Table structure */
typedef struct {
    int id;                /* e.g., 1 for T01 */
    int capacity;          /* e.g., 2, 4, 6 */
    TableStatus status;
    int current_order_id;  /* Order ID occupying this table, or 0 */
    char reserved_for[50]; /* Name for reservation, if RESERVED */
} Table;

/* Order Item structure */
typedef struct {
    FoodItem item;
    int quantity;
    double amount;      /* price * quantity */
} OrderItem;

/* Order structure */
typedef struct {
    int order_id;
    int table_id;       /* 0 if walk-in / no table */
    char customer_name[50];
    OrderItem items[MAX_ORDER_ITEMS];
    int item_count;
    OrderStatus status;
    double subtotal;
    double discount_rate;
    double discount_amount;
    double tax_rate;
    double tax_amount;
    double grand_total;
    char timestamp[30];
} Order;
```

These four structures form the backbone of the data model. FoodItem represents a menu entry; Table represents a physical table and its live status; OrderItem represents one line of an order; and Order aggregates a customer's items along with billing fields and a timestamp.

Snippet 2 — Status Enumerations (TableStatus, OrderStatus)

```
/* Table Statuses */
typedef enum {
    TABLE_AVAILABLE = 0,
```

```
    TABLE_OCCUPIED,
    TABLE_RESERVED
} TableStatus;

/* Order Statuses */
typedef enum {
    STATUS_PENDING = 0,
    STATUS_PREPARING,
    STATUS_READY,
    STATUS_SERVED,
    STATUS_COMPLETED,
    STATUS_CANCELLED
} OrderStatus;
```

TableStatus defines the three states a table can be in (Available, Occupied, Reserved), and OrderStatus defines the six states an order moves through, from Pending at creation to Completed or Cancelled at closure. These enumerations drive the Table Management and Kitchen Display System screens.

Snippet 3 — Dashboard and Reporting Structures

```
/* Dashboard & Reports Statistics */
typedef struct {
    int total_orders;
    int active_orders;
    int completed_orders;
    int pending_orders;
    int preparing_orders;
    int ready_orders;
    int served_orders;
    int cancelled_orders;
    int available_tables;
    int occupied_tables;
    int reserved_tables;
    double today_revenue;
    double avg_order_value;
    double occupancy_rate; /* percentage e.g. 40.0% */
} DashboardStats;

typedef struct {
    int item_id;
    char item_name[60];
    int total_quantity_sold;
    double total_revenue_generated;
} TopItemStat;
```

DashboardStats aggregates order counts by status, table counts by status, today's revenue, average order value and occupancy rate for the Dashboard screen. TopItemStat holds the quantity sold and revenue generated per menu item, used by the Reports module to identify top-selling items.

Snippet 4 — Billing Constants and Order/Business-Logic Interface

```
#define DISCOUNT_RATE 0.05  /* 5% Discount */
#define TAX_RATE 0.05       /* 5% GST Tax */

/* Order API (business-logic interface declared in restaurant.h) */
int createOrder(int table_id, const char *customer_name,
                char *err_msg, size_t err_size);
int addItemToOrder(Order *order, int item_id, int quantity,
                   char *err_msg, size_t err_size);
int updateItemQuantity(Order *order, int item_id, int new_quantity,
                       char *err_msg, size_t err_size);
```

```
int removeItemFromOrder(Order *order, int item_id,
                        char *err_msg, size_t err_size);
int updateOrderStatus(int order_id, OrderStatus status,
                      char *err_msg, size_t err_size);
void calculateTotals(Order *order);

/* Metrics & Helper API */
void getDashboardStats(DashboardStats *stats);
int getTopSellingItems(TopItemStat *top_items, int max_items);
```

DISCOUNT_RATE and TAX_RATE define the 5% discount and 5% tax applied during bill calculation. The declared functions form the interface used to create orders, add or update items within an order, change an order's status, compute totals via calculateTotals(), and retrieve dashboard and top-selling-item statistics. These declarations are implemented in restaurant.c and invoked from the GTK3 callbacks in gui.c.

15. TESTING

The application's modules were tested using representative sample data to verify that each function behaves as intended. The table below summarizes the test cases exercised during development.

Test ID	Module	Test Action	Expected Result	Actual Result	Status
TC01	Application	Launch the application	Dashboard loads with statistics	As expected	Pass
TC02	Dashboard	Load dashboard on startup	Order and table statistics are displayed	As expected	Pass
TC03	Menu	View the menu list	All configured menu items are displayed with price and status	As expected	Pass
TC04	Menu	Search for a menu item by name	Matching items are shown	As expected	Pass
TC05	Menu	Filter menu by category	Only items in the selected category are shown	As expected	Pass
TC06	Menu	Enable/disable a menu item	Item availability toggles and reflects in the list	As expected	Pass
TC07	Tables	View table availability	Each table shows correct status (Available/Occupied/Reserved)	As expected	Pass
TC08	Tables	Reserve an available table	Table status changes to Reserved with customer name	As expected	Pass
TC09	Tables	Assign an available table	Table status changes to Occupied and links to the order	As expected	Pass
TC10	Orders	Create a new order for a table	Order is created with a unique order ID	As expected	Pass
TC11	Orders	Add an item to an order	Item appears in the order with computed amount	As expected	Pass
TC12	Orders	Update item quantity	Amount recalculates for the updated quantity	As expected	Pass
TC13	Orders	Remove an item from an order	Item is removed and totals recalculate	As expected	Pass
TC14	Kitchen	Update order status through kitchen stages	Status progresses Pending → Preparing → Ready → Served	As expected	Pass
TC15	Billing	Generate a bill for a served order	Subtotal, 5% discount, 5% tax and grand total are correct	As expected	Pass
TC16	Dashboard	Check dashboard statistics after order activity	Totals and occupancy update accordingly	As expected	Pass
TC17	Reports	Check top-selling item calculation	Items are ranked by quantity sold and revenue	As expected	Pass
TC18	Data	Save/reload restaurant data (where implemented)	Data is retained across the operation	As expected	Pass

16. APPLICATION SCREENSHOTS AND WORKING

The following screenshots are taken directly from the working Restaurant Management System application (branded in the demonstration build as "SPICE BAVARCHI RESTAURANT"). The figures and sample values visible in these screenshots are demonstration data used to illustrate the application's behaviour and should not be interpreted as real restaurant business statistics.

Figure 1: Restaurant Executive Dashboard and Analytics

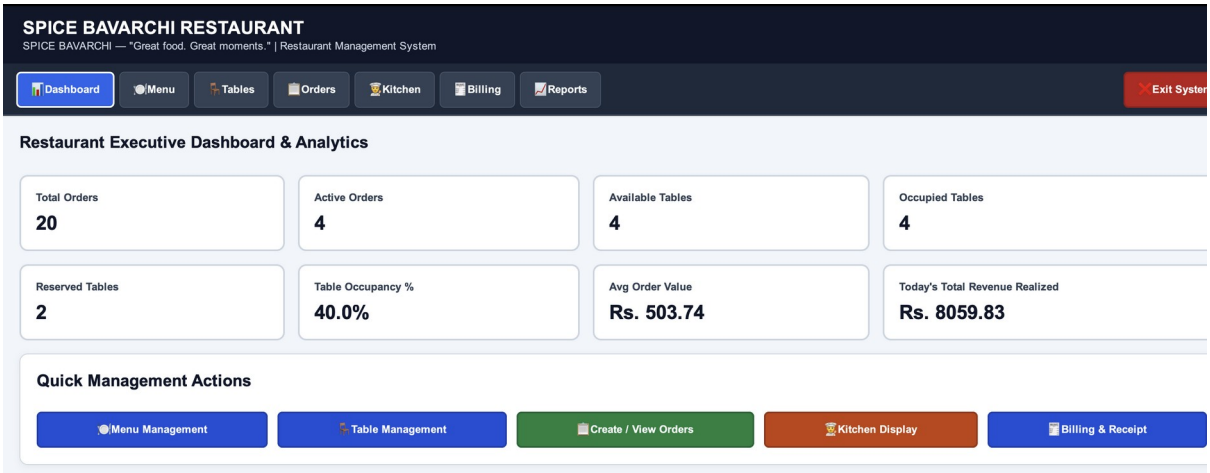

Figure 1: The Dashboard screen shows Total Orders, Active Orders, Available/Occupied/Reserved Tables, Table Occupancy %, Average Order Value, Today's Revenue, and Quick Management Action shortcuts.

The dashboard gives staff and management a single-glance operational overview and one-click navigation into the Menu, Table, Order, Kitchen and Billing modules. The values shown (e.g., 20 total orders, 40.0% occupancy, Rs. 8059.83 revenue) are sample data generated to demonstrate the screen and do not represent an actual restaurant's real trading figures.

Figure 2: Table Management and Floor Seating Overview

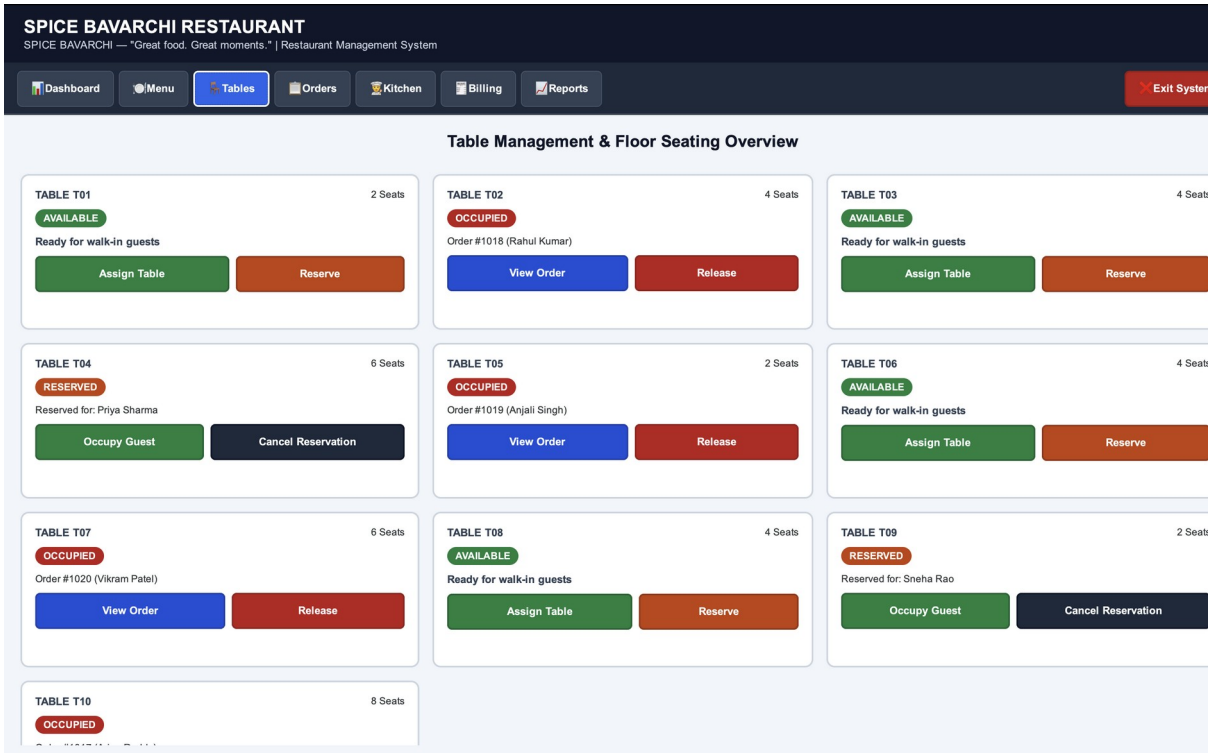

Figure 2: The Table Management screen lists tables T01–T10 with seating capacity, live status, and context-appropriate actions (Assign Table, Reserve, View Order, Release, Occupy Guest, Cancel Reservation).

Each table card reflects its current TableStatus. Occupied tables show the linked order number and customer name with a View Order / Release action; Reserved tables show the reservation name with Occupy Guest / Cancel Reservation actions; Available tables offer Assign Table / Reserve actions. The table numbers, seat counts, and customer names shown are demonstration data.

Figure 3: Restaurant Menu Management

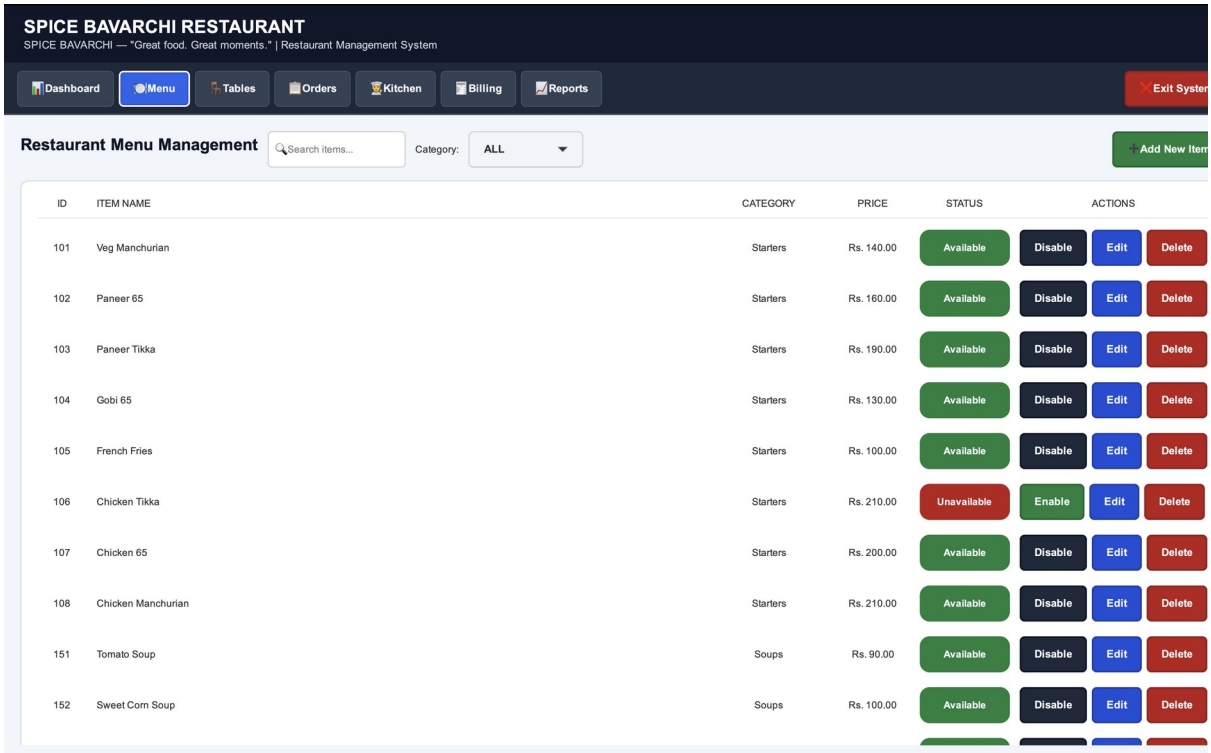

Figure 3: The Menu Management screen lists menu items with ID, name, category, price and availability, alongside search, category filtering and Add New Item controls.

Staff can search by name, filter by category, and Enable/Disable, Edit or Delete individual items. The screenshot shows starter items such as Veg Manchurian (Rs. 140.00), Paneer 65 (Rs. 160.00), Paneer Tikka (Rs. 190.00), Gobi 65 (Rs. 130.00), French Fries (Rs. 100.00), Chicken Tikka (Rs. 210.00, shown Unavailable), Chicken 65 (Rs. 200.00) and Chicken Manchurian (Rs. 210.00), together with soups such as Tomato Soup (Rs. 90.00) and Sweet Corn Soup (Rs. 100.00). These items and prices are as configured in the demonstration build of the application.

17. ADVANTAGES

- Centralized management of restaurant operations in a single desktop application.
- Graphical, click-driven user interface built with GTK3, replacing manual paperwork.
- Structured menu management with search, filtering and availability control.
- Clear, at-a-glance table management with reservation support.
- Organized order management with itemized quantities and amounts.
- Kitchen order tracking through defined preparation stages, reducing communication gaps.
- Faster, more consistent billing with automatic discount and tax calculation.
- Reduced manual calculation errors compared to hand-written bills.
- Dashboard analytics for a quick operational overview.
- Top-selling item analysis to support menu and business decisions.
- Modular C implementation that separates business logic from the GUI layer.
- Easier day-to-day operational monitoring for restaurant staff and management.

18. LIMITATIONS

- The application is a desktop/local system and is not accessible remotely.
- There is no online ordering facility for customers.
- There is no online or digital payment gateway integration.
- There is no cloud synchronization of data.
- There is no multi-device or multi-terminal synchronization.
- The screenshots and figures used for demonstration in this report are sample data rather than real trading data.
- Scalability is limited compared with enterprise-grade commercial restaurant platforms.
- Advanced inventory management is not implemented in the current build.
- There is no dedicated customer-facing mobile application.

19. FUTURE SCOPE

The following enhancements are proposed as possible future improvements and are not part of the current implementation:

- Integration with a persistent, production-grade database.
- User authentication and role-based access control for staff and management.
- Inventory management for tracking ingredient and stock levels.
- Customer management and loyalty tracking.
- Supplier management for procurement.
- Online ordering for customers.
- Digital/online payment integration.
- Cloud synchronization across devices and locations.
- Multi-device and multi-terminal support for larger restaurants.
- Advanced analytics and trend reporting.
- Automated kitchen notifications.
- A companion mobile application.
- A web-based administration panel.

20. CONCLUSION

The Restaurant Management System demonstrates how the C programming language, using the C99 standard together with the GTK3 GUI toolkit, can be applied to build a practical desktop application for managing the full range of restaurant operations rather than billing alone. Through the integrated Dashboard, Menu Management, Table Management, Order Management, Kitchen Display System, Billing, and Reports/Analytics modules, the project shows how structured data modelling, modular programming and event-driven GUI development can be combined to solve a real operational problem.

The project demonstrates practical use of C structures, enumerations, arrays, functions, conditional logic and loops alongside GTK3-based GUI programming and callback-driven event handling. It shows that a well-organized desktop application can meaningfully reduce manual effort, improve coordination between front-of-house and kitchen staff, and provide clearer operational visibility than a purely manual or paper-based approach, while also serving as a substantial demonstration of the programming concepts covered in the B.Tech curriculum.

21. REFERENCES

Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, Prentice Hall.

E. Balagurusamy, Programming in ANSI C, McGraw-Hill.

Yashavant Kanetkar, Let Us C, BPB Publications.

Herbert Schildt, C: The Complete Reference, McGraw-Hill.

GTK3 Developer Documentation, GNOME Project (docs.gtk.org).

Standard C (C99) Language Reference and academic resources on structured and modular programming, and event-driven GUI application development.